## READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))


all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")
```

```
print(f"pneumon : {len(all_pneumonia_files)}")
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

```
train_pneumonia_samples = random_select_files(all_pneumonia_files, 1181)
test_pneumonia_samples = random_select_files([f for f in all_pneumonia_files if f not in train_pneumonia_samples], 394)

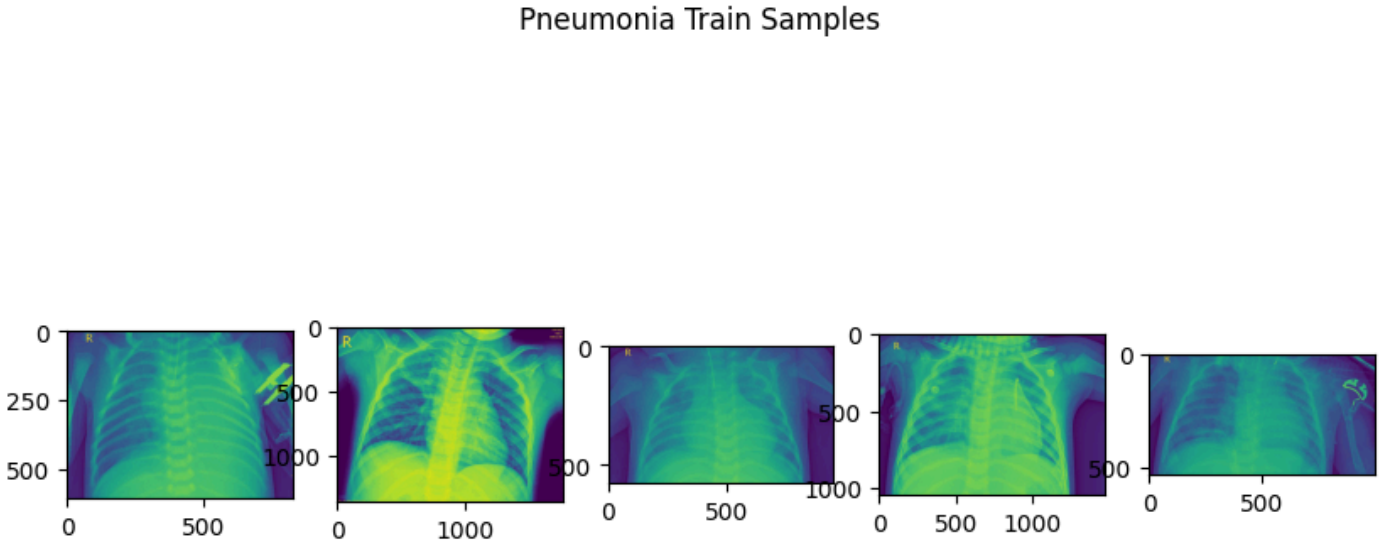
train_normal_samples = random_select_files(all_normal_files, 1181)
test_normal_samples = random_select_files([f for f in all_normal_files if f not in train_normal_samples], 394)

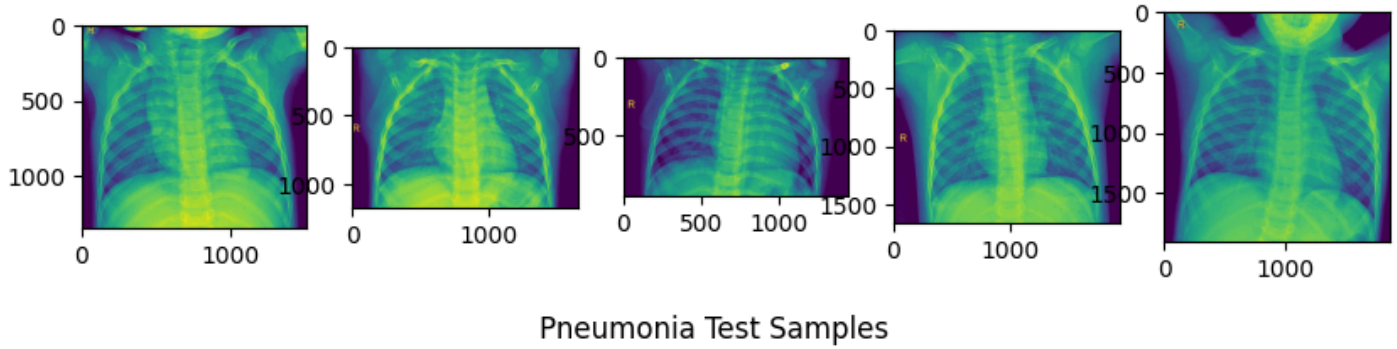
#train_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in train_pneumonia_files]
#test_pneumonia_samples = [os.path.join(train_pneumonia_dir, file) for file in test_pneumonia_files]
#train_normal_samples = [os.path.join(train_normal_dir, file) for file in train_normal_files]
#test_normal_samples = [os.path.join(train_normal_dir, file) for file in test_normal_files]
```

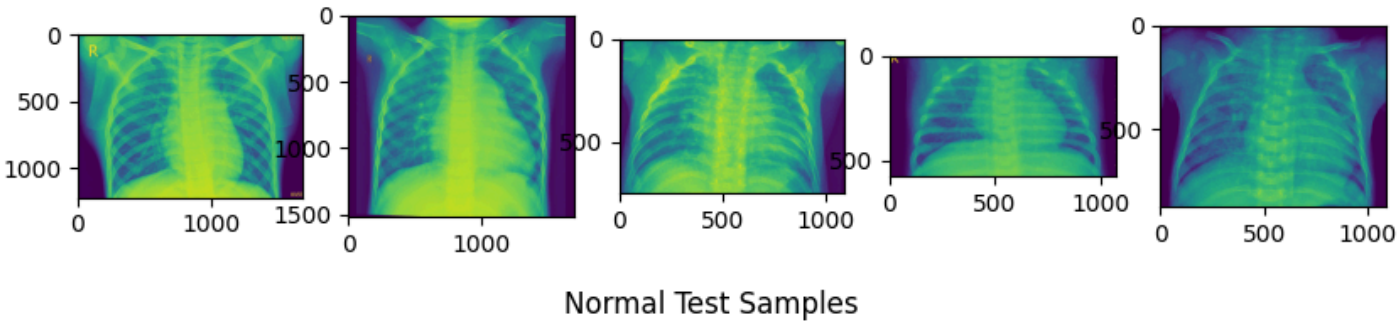
```
from PIL import Image
import matplotlib.pyplot as plt

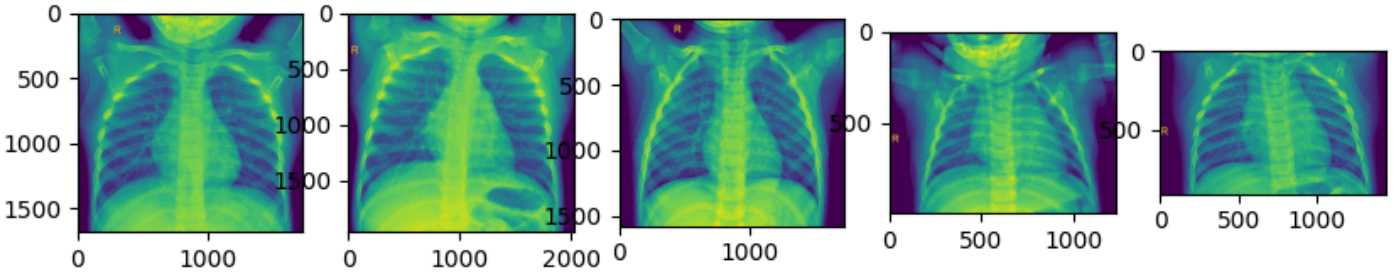
def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
        img = Image.open(sample_files[i])
        plt.imshow(img)
    plt.show()

show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```


Pneumonia Train Samples
Normal Train Samples


Pneumonia Test Samples


Normal Test Samples



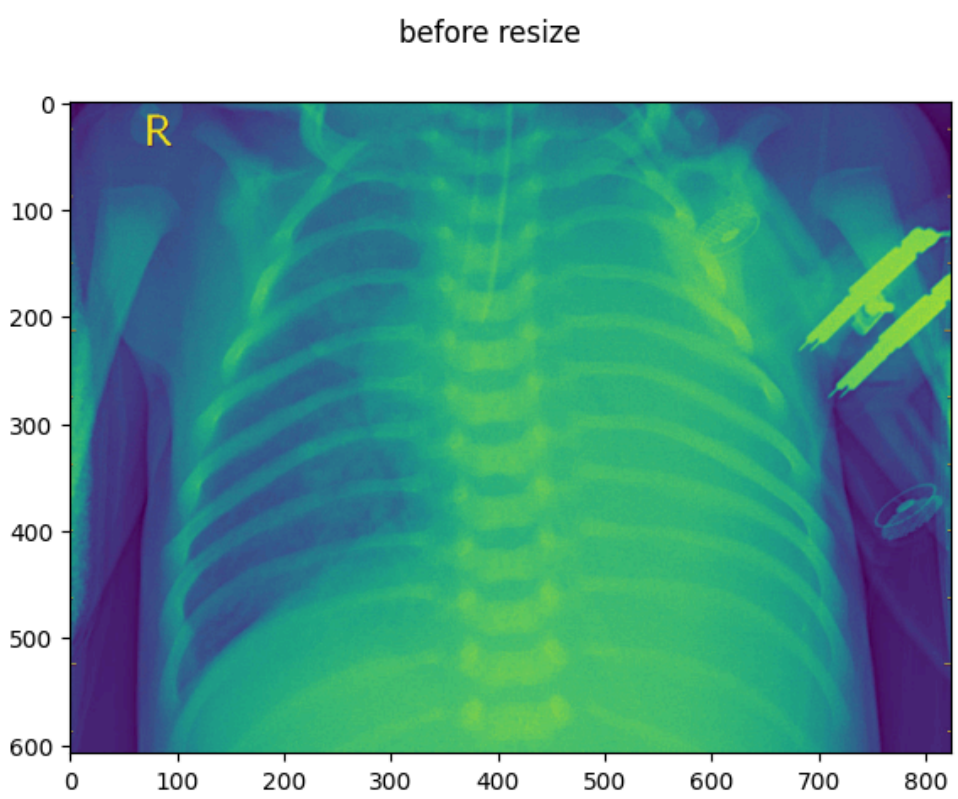
## RESIZE

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

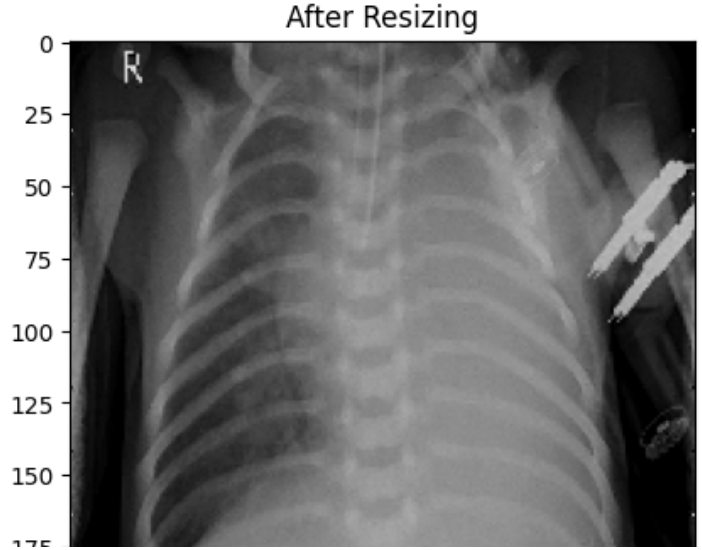
def load_images(file_paths, target_size=(216, 216)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

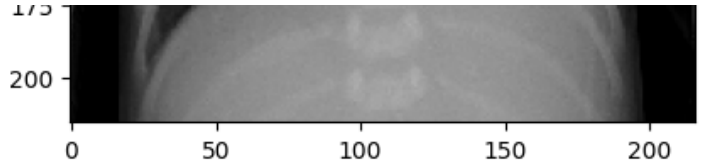
train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)


show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```




```
WARNING:matplotlib.image:Clipping input data to the valid range for imshow with RGB data ([0..1] for floats or [0..255] for integers). Got range [-0.1176
Text(0.5, 1.0, 'After Resizing')
```






## GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```

```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
        glcm_features_list.append(features)
    return glcm_features_list

train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
train_normal_glcm = extract_glcm_features(train_normal_images)
test_normal_glcm = extract_glcm_features(test_normal_images)

print("GLCM features for the first pneumonia training image:")
train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(1176.8476730741068),
 'contrast_std': np.float64(478.8529863589346),
 'contrast_var': np.float64(229300.18254486998),
 'dissimilarity_mean': np.float64(11.366615409452237),
 'dissimilarity_std': np.float64(3.486739285901238),
 'dissimilarity_var': np.float64(12.157350847847075),
 'homogeneity_mean': np.float64(0.1895484843859004),
 'homogeneity_std': np.float64(0.03468601342472142),
 'homogeneity_var': np.float64(0.0012031195272999547),
 'energy_mean': np.float64(0.018651575880244353),
 'energy_std': np.float64(0.0016530228510250984),
 'energy_var': np.float64(2.732484546011144e-06),
 'correlation_mean': np.float64(0.8048631211786755),
 'correlation_std': np.float64(0.08034552937448382),
 'correlation_var': np.float64(0.006455404090466041),
 'ASM_mean': np.float64(0.0003506137673625239),
 'ASM_std': np.float64(6.216375836694336e-05),
 'ASM_var': np.float64(3.86433285430372e-09),
 'entropy': np.float64(147.8796372404704)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'
```

```
#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'

#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

## CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)
```

```
# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=35, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.3,
    callbacks=None,
    class_weight=class_weights,
)
```

```
/usr/local/lib/python3.11/dist-packages/keras/src/layers/core/dense.py:87: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. Whe
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential"
```

| Layer (type) | Output Shape | Param # |
|---|---|---|
| dense (Dense) | (None, 128) | 2,560 |
| batch_normalization (BatchNormalization) | (None, 128) | 512 |
| dropout (Dropout) | (None, 128) | 0 |
| dense_1 (Dense) | (None, 1) | 129 |

```
 Total params: 3,201 (12.50 KB)
 Trainable params: 2,945 (11.50 KB)
 Non-trainable params: 256 (1.00 KB)
Epoch 1/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 4s 11ms/step - accuracy: 0.6720 - auc: 0.7496 - loss: 0.6333 - val_accuracy: 0.7645 - val_auc: 0.0000e+00 - val_loss: 0.6242
Epoch 2/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8620 - auc: 0.9288 - loss: 0.3570 - val_accuracy: 0.6770 - val_auc: 0.0000e+00 - val_loss: 0.6821
Epoch 3/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8723 - auc: 0.9292 - loss: 0.3355 - val_accuracy: 0.6869 - val_auc: 0.0000e+00 - val_loss: 0.6800
Epoch 4/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8837 - auc: 0.9404 - loss: 0.3125 - val_accuracy: 0.6770 - val_auc: 0.0000e+00 - val_loss: 0.6867
Epoch 5/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8824 - auc: 0.9446 - loss: 0.2994 - val_accuracy: 0.7673 - val_auc: 0.0000e+00 - val_loss: 0.5952
Epoch 6/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8770 - auc: 0.9347 - loss: 0.3252 - val_accuracy: 0.8364 - val_auc: 0.0000e+00 - val_loss: 0.4644
Epoch 7/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8902 - auc: 0.9525 - loss: 0.2792 - val_accuracy: 0.8068 - val_auc: 0.0000e+00 - val_loss: 0.5377
Epoch 8/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.8913 - auc: 0.9365 - loss: 0.3174 - val_accuracy: 0.8674 - val_auc: 0.0000e+00 - val_loss: 0.4269
Epoch 9/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8957 - auc: 0.9486 - loss: 0.2902 - val_accuracy: 0.8632 - val_auc: 0.0000e+00 - val_loss: 0.4009
Epoch 10/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8935 - auc: 0.9492 - loss: 0.2849 - val_accuracy: 0.8674 - val_auc: 0.0000e+00 - val_loss: 0.3859
Epoch 11/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8838 - auc: 0.9452 - loss: 0.2942 - val_accuracy: 0.8660 - val_auc: 0.0000e+00 - val_loss: 0.4038
Epoch 12/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8893 - auc: 0.9457 - loss: 0.2928 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3639
Epoch 13/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8936 - auc: 0.9458 - loss: 0.3052 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3760
Epoch 14/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8927 - auc: 0.9470 - loss: 0.2859 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3894
Epoch 15/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8860 - auc: 0.9514 - loss: 0.2776 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3495
Epoch 16/50
```

```
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8891 - auc: 0.9441 - loss: 0.2966 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3557
Epoch 17/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.8988 - auc: 0.9540 - loss: 0.2726 - val_accuracy: 0.8886 - val_auc: 0.0000e+00 - val_loss: 0.3540
Epoch 18/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8894 - auc: 0.9443 - loss: 0.3002 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3396
Epoch 19/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8839 - auc: 0.9508 - loss: 0.2807 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3683
Epoch 20/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8937 - auc: 0.9540 - loss: 0.2675 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3518
Epoch 21/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9030 - auc: 0.9599 - loss: 0.2503 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3700
Epoch 22/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8921 - auc: 0.9554 - loss: 0.2630 - val_accuracy: 0.8674 - val_auc: 0.0000e+00 - val_loss: 0.4018
Epoch 23/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.9076 - auc: 0.9617 - loss: 0.2444 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3760
Epoch 24/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8934 - auc: 0.9608 - loss: 0.2511 - val_accuracy: 0.8970 - val_auc: 0.0000e+00 - val_loss: 0.3240
Epoch 25/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9048 - auc: 0.9559 - loss: 0.2627 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3459
Epoch 26/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9098 - auc: 0.9631 - loss: 0.2409 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3556
Epoch 27/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9104 - auc: 0.9630 - loss: 0.2424 - val_accuracy: 0.8731 - val_auc: 0.0000e+00 - val_loss: 0.3547
Epoch 28/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9138 - auc: 0.9622 - loss: 0.2437 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3490
Epoch 29/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9009 - auc: 0.9653 - loss: 0.2386 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3471
Epoch 30/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9086 - auc: 0.9574 - loss: 0.2569 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3343
Epoch 31/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - accuracy: 0.8946 - auc: 0.9514 - loss: 0.2852 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3388
Epoch 32/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9057 - auc: 0.9677 - loss: 0.2285 - val_accuracy: 0.8815 - val_auc: 0.0000e+00 - val_loss: 0.3614
Epoch 33/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9069 - auc: 0.9590 - loss: 0.2511 - val_accuracy: 0.8759 - val_auc: 0.0000e+00 - val_loss: 0.3677
Epoch 34/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8990 - auc: 0.9604 - loss: 0.2538 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3522
Epoch 35/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9167 - auc: 0.9654 - loss: 0.2364 - val_accuracy: 0.8900 - val_auc: 0.0000e+00 - val_loss: 0.3423
Epoch 36/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - accuracy: 0.8932 - auc: 0.9561 - loss: 0.2661 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3420
Epoch 37/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9093 - auc: 0.9647 - loss: 0.2385 - val_accuracy: 0.8590 - val_auc: 0.0000e+00 - val_loss: 0.4050
Epoch 38/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8921 - auc: 0.9607 - loss: 0.2504 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3436
Epoch 39/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9174 - auc: 0.9691 - loss: 0.2186 - val_accuracy: 0.8759 - val_auc: 0.0000e+00 - val_loss: 0.3688
Epoch 40/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9123 - auc: 0.9659 - loss: 0.2263 - val_accuracy: 0.8858 - val_auc: 0.0000e+00 - val_loss: 0.3419
```

```
Epoch 41/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9057 - auc: 0.9680 - loss: 0.2240 - val_accuracy: 0.8942 - val_auc: 0.0000e+00 - val_loss: 0.3370
Epoch 42/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9130 - auc: 0.9675 - loss: 0.2282 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3555
Epoch 43/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9030 - auc: 0.9587 - loss: 0.2569 - val_accuracy: 0.8914 - val_auc: 0.0000e+00 - val_loss: 0.3177
Epoch 44/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9031 - auc: 0.9622 - loss: 0.2458 - val_accuracy: 0.8928 - val_auc: 0.0000e+00 - val_loss: 0.3212
Epoch 45/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.8916 - auc: 0.9528 - loss: 0.2771 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3333
Epoch 46/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9021 - auc: 0.9617 - loss: 0.2461 - val_accuracy: 0.8843 - val_auc: 0.0000e+00 - val_loss: 0.3422
Epoch 47/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9104 - auc: 0.9627 - loss: 0.2442 - val_accuracy: 0.8745 - val_auc: 0.0000e+00 - val_loss: 0.3582
Epoch 48/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9128 - auc: 0.9661 - loss: 0.2324 - val_accuracy: 0.8829 - val_auc: 0.0000e+00 - val_loss: 0.3409
Epoch 49/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - accuracy: 0.9041 - auc: 0.9668 - loss: 0.2340 - val_accuracy: 0.8787 - val_auc: 0.0000e+00 - val_loss: 0.3378
Epoch 50/50
52/52 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - accuracy: 0.9158 - auc: 0.9701 - loss: 0.2157 - val_accuracy: 0.8801 - val_auc: 0.0000e+00 - val_loss: 0.3483
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step
- Loss: 0.2736
- AUC: 0.9591
- Accuracy: 0.8985
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.92      0.88      0.90       394
           1       0.88      0.92      0.90       394

    accuracy                           0.90       788
   macro avg       0.90      0.90      0.90       788
weighted avg       0.90      0.90      0.90       788
```

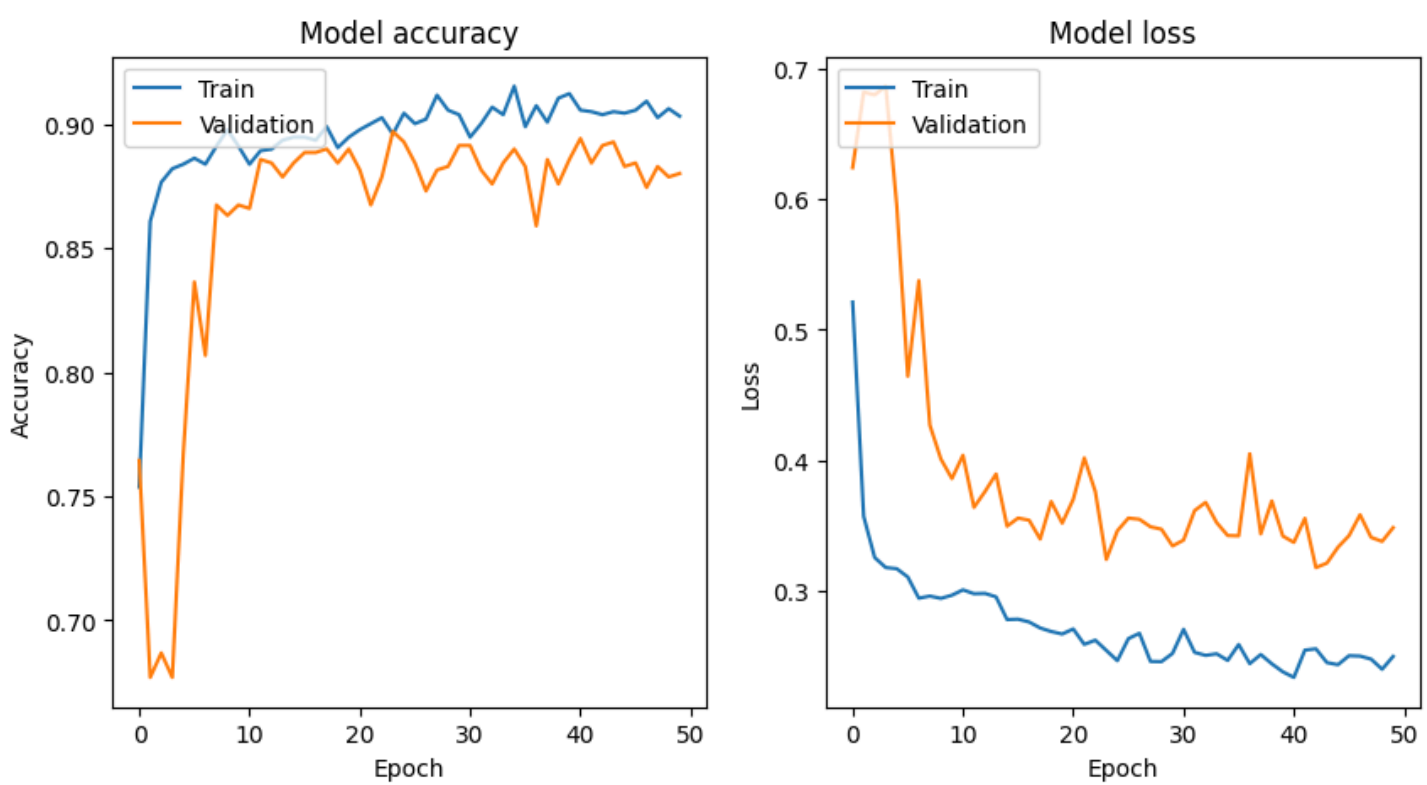
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
```

```
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```




## PREDICT

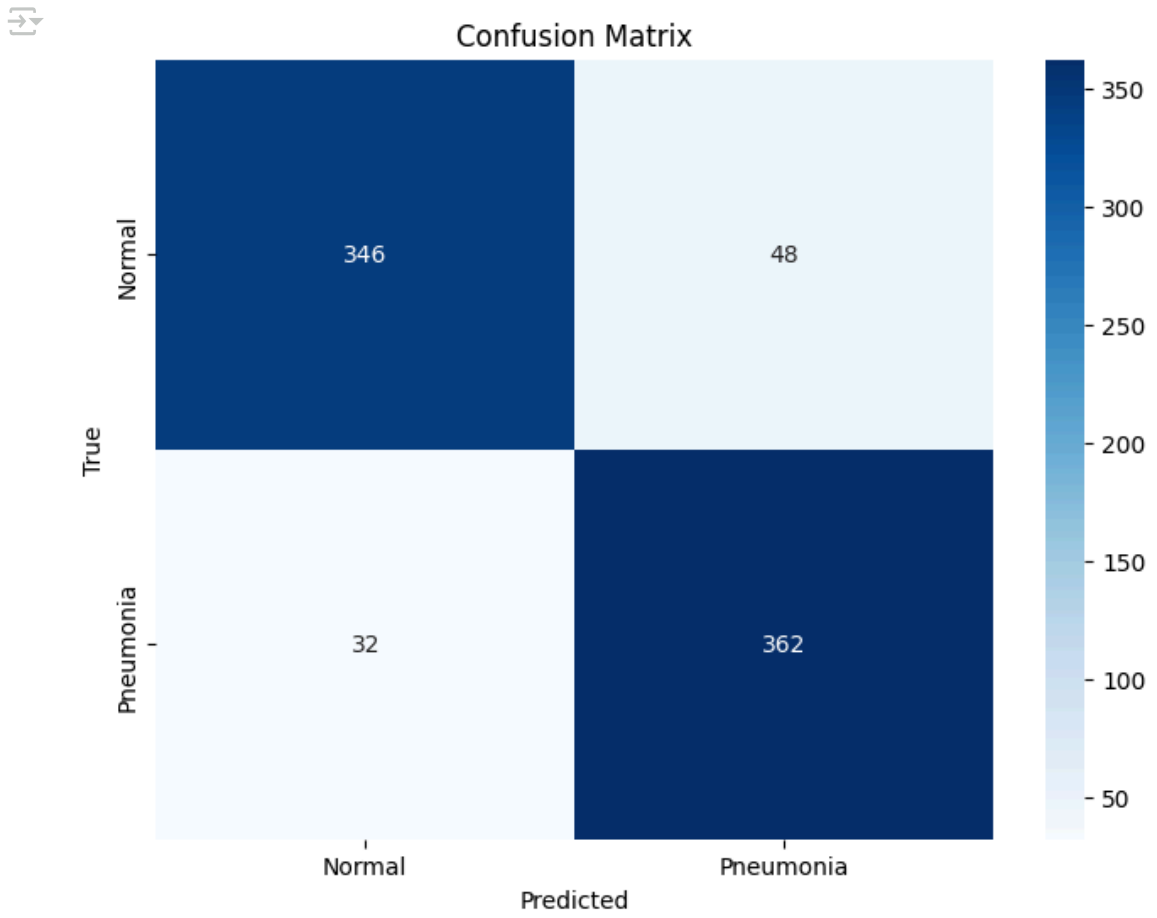
```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

```
25/25 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```



```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/NORMAL'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
```

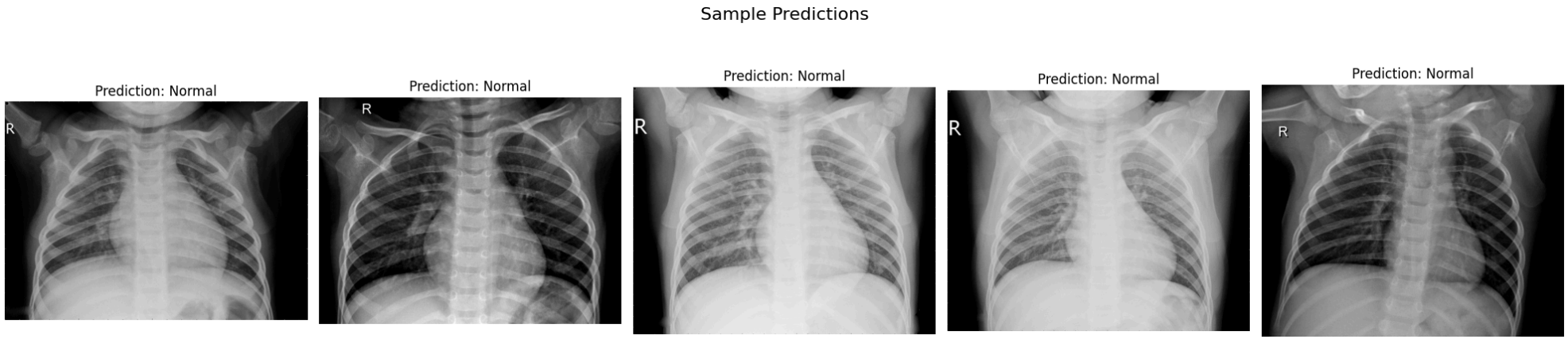
```
            axes[i].set_title(f"Prediction: {prediction_for_sample}")
            axes[i].axis('off')

        except Exception as e:
            print(f"Error processing {image_file}: {e}")

    plt.tight_layout()
    plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 34ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 61ms/step
```

| | Image Path | Prediction |
|---|---|---|
| 0 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 1 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 2 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 3 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 4 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 5 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 6 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |
| 7 | /content/drive/MyDrive/TA/Dataset/val/NORMAL/N... | Normal |




```
import matplotlib.pyplot as plt
import cv2
import pandas as pd
import os
```

```
import random

test_dir = '/content/drive/MyDrive/TA/Dataset/val/PNEUMONIA'
image_files = [os.path.join(test_dir, f) for f in os.listdir(test_dir) if os.path.isfile(os.path.join(test_dir, f))]

image_paths = []
predictions = []

for image_file in image_files:
    try:
        new_image = load_img(image_file, target_size=(150, 150))
        new_image_array = img_to_array(new_image)
        new_image_array = np.clip(new_image_array - 30, 0, 255)
        new_image_array = new_image_array / 255.0
        new_image_glcm = extract_glcm_features([new_image_array])
        new_image_glcm_df = pd.DataFrame(new_image_glcm)
        new_image_scaled = scaler.transform(new_image_glcm_df)
        prediction = model.predict(new_image_scaled)
        predicted_class = (prediction > 0.5).astype(int)[0][0]

        image_paths.append(image_file)
        predictions.append('Pneumonia' if predicted_class == 1 else 'Normal')

    except Exception as e:
        print(f"Error processing {image_file}: {e}")

results_df = pd.DataFrame({'Image Path': image_paths, 'Prediction': predictions})
print(results_df)

sample_files = random.sample(image_files, 5)

fig, axes = plt.subplots(1, 5, figsize=(20, 5))
fig.suptitle('Sample Predictions', fontsize=16)

for i, image_file in enumerate(sample_files):
    try:
        img = cv2.imread(image_file, cv2.IMREAD_GRAYSCALE)

        axes[i].imshow(img, cmap='gray')
        prediction_for_sample = results_df[results_df['Image Path'] == image_file]['Prediction'].values[0]
        axes[i].set_title(f"Prediction: {prediction_for_sample}")
        axes[i].axis('off')

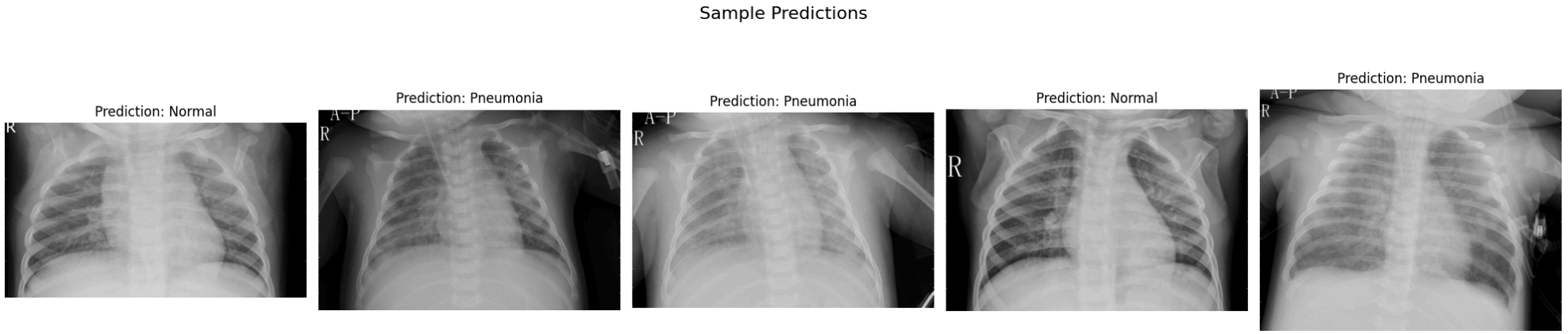
    except Exception as e:
```

```
        print(f"Error processing {image_file}: {e}")

plt.tight_layout()
plt.show()
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 36ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 38ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 41ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 56ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 42ms/step
```

| | Image Path | Prediction |
|---|---|---|
| 0 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Normal |
| 1 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Normal |
| 2 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Pneumonia |
| 3 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Normal |
| 4 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Normal |
| 5 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Normal |
| 6 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Pneumonia |
| 7 | /content/drive/MyDrive/TA/Dataset/val/PNEUMONI... | Pneumonia |




```
# prompt: uji data glcm ini juga dengan svm

from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)
```

```
# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

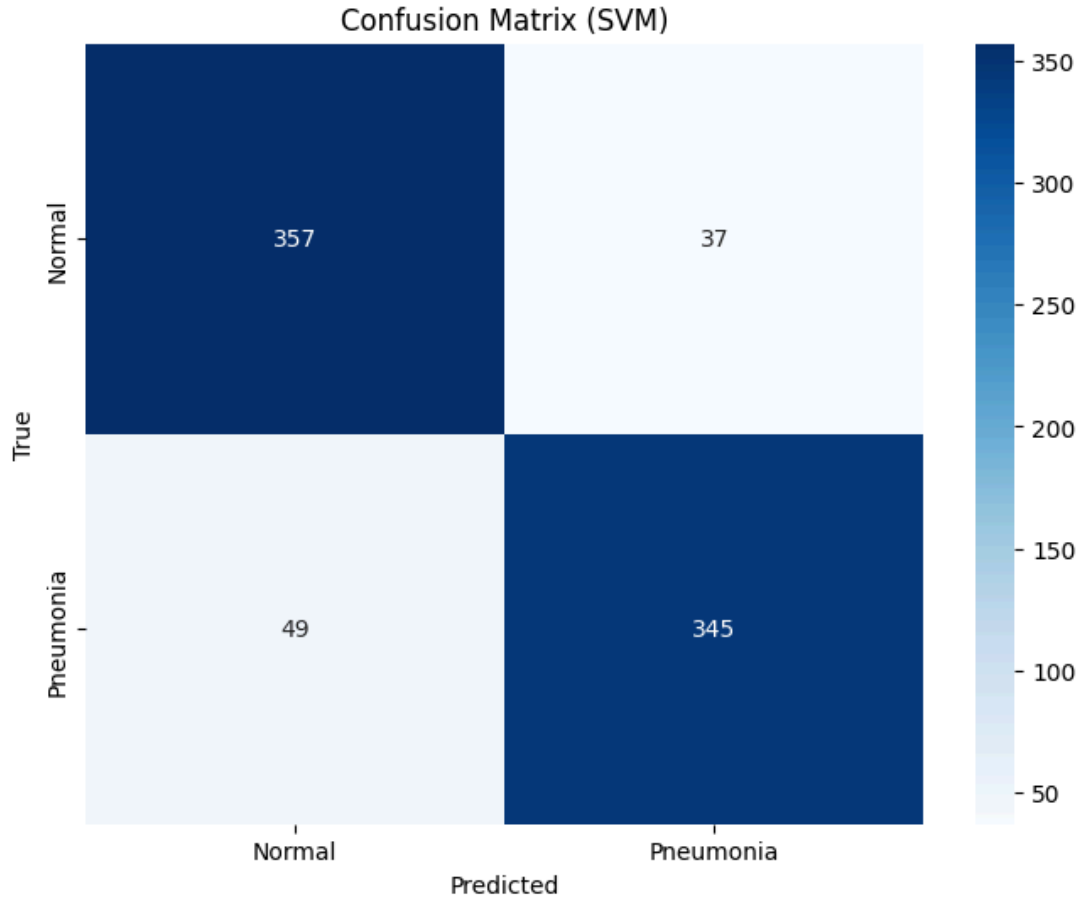
# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```

```
--- SVM Model Evaluation ---
Accuracy: 0.8909

Classification Report:
              precision    recall  f1-score   support

           0       0.88      0.91      0.89       394
           1       0.90      0.88      0.89       394

    accuracy                           0.89       788
   macro avg       0.89      0.89      0.89       788
weighted avg       0.89      0.89      0.89       788
```



```
# prompt: uji juga dengan random forest

from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

# Create and train the Random Forest model
# You can adjust n_estimators (number of trees) and other parameters
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_rf = rf_model.predict(X_test)

# Evaluate the Random Forest model
print("\n--- Random Forest Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_rf):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_rf))

# Confusion Matrix
conf_matrix_rf = confusion_matrix(y_test, y_pred_rf)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_rf, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (Random Forest)')
plt.show()
```

```
--- Random Forest Model Evaluation ---
Accuracy: 0.9086

Classification Report:
              precision    recall  f1-score   support

           0       0.90      0.92      0.91       394
           1       0.92      0.90      0.91       394

    accuracy                           0.91       788
   macro avg       0.91      0.91      0.91       788
weighted avg       0.91      0.91      0.91       788
```

